

SiliconLoop

在 ESP32-P4 上让 AI Agent 自主诊断硬件故障

2026 首届 openvela AI 硬件开发者大赛 · 队伍 330 · 内核游牧者

一、问题背景

嵌入式设备出故障时，通常没有人在旁边。

设备装在墙上、车里、工厂产线上，串口没接，日志没人看。等到有人发现“这台不对劲”，往往已经过了几天，现场早就没了。工程师能做的是把设备拆回来、接上串口、祈祷问题能复现。

更麻烦的是有一类故障**根本不会报错**。

```
while (!fifo_has_space())
    usleep(1000);
fifo_put(g_produced++);
```

这是一段生产者代码：等缓冲区有空间，然后写入。如果消费者停止消费，缓冲区填满，`fifo_has_space()` 永远返回 `false`，这个循环就永远转下去。

它不崩溃、不报错、不打日志。进程还在，CPU 占用正常，从外面看一切正常——只是再也不往前走了。

这类故障在测试环境里发现不了。单元测试跑的是 `mock`——用一个假的缓冲区替代真实硬件，`fifo_has_space()` 恒返回 `true`，没有容量限制，测试永远通过。只有在真机上，缓冲区真的只有 64 字节，真的会填满。

需要说清楚的是：正常运行时缓冲区也会填满。生产者每 10 毫秒放一个字节，消费者每 20 毫秒取一个，生产比消费快一倍，必然会满。但满了之后消费者不断取走、腾出空间，生产者等一下就能继续——整个系统被拖慢到消费者的速度，**但一直在前进**。

故障不是“缓冲区满了”，而是“满了之后再也不会空”。

SiliconLoop 的做法是：让 AI Agent 跑在设备上，每两分钟自己醒来一次，采集系统状态，判断任务是否还在前进，把结论和证据写成报告。不需要人在现场，不需要接串口。

这件事的难点不在于“调用一个大模型”，而在于**让它在 404 KB 堆、无 RTC、单核 RISC-V 的设备上稳定工作五个小时以上**。第三节的回归数据和第四节的技术难点，讲的都是这件事。

二、技术方案

项目有两条主线，它们本来是一件事的两半。

诊断解决“设备出了什么问题”，**动态加载**解决“修复怎么推下去”——诊断出问题之后，把修好的程序推到设备上运行，不用重新烧录整块固件。

第二条主线最终只做到了“板子能加载并执行外部编译的程序”，没有做推送通道。但正是这一步暴露出 ESP32-P4 在 openvela 上根本不支持 ELF 动态加载，两层根因都在平台层，与本项目无关——任何需要运行时加载模块的应用都会撞上。

主线一：Agent 心跳自诊断

AI Agent 运行在 ESP32-P4 上，每 120 秒被心跳服务自动唤醒，执行一次七步诊断：

步骤	动作	目的
1	<code>read_file HEARTBEAT.md</code>	获取本轮任务描述
2	<code>read_file self-diagnostic.md</code>	加载诊断 Skill
3	<code>read_file heartbeat.txt</code>	第一次采样
4	<code>run_shell "ps"</code>	进程列表
5	<code>run_shell "free"</code>	堆内存状态
6	<code>read_file heartbeat.txt</code>	第二次采样
7	<code>write_file report.md</code>	写出四段式报告

核心判据是两次采样的差值。 被监控的任务每生产 50 个数据，就往 heartbeat.txt 写一个递增的计数。计数增加说明任务在前进，不变说明卡住了。

为什么中间要插 ps 和 free

两次采样之间必须隔开足够长的时间，否则会误判——连着读两次，程序还没来得及更新计数，数字自然不变。

但 Agent 的工具集里**没有 sleep**，它无法主动等待。步骤 4、5 就是为此设计的：每一次工具调用都要经过一轮 LLM 往返，两条命令自然拉开十余秒。用别的工作填满间隔，是绕过这个限制的办法。

这两条命令的输出本身也有用——报告里会引用，让结论更有依据。

为什么判据是心跳而不是进程列表

ps 在正常和故障状态下看到的完全相同。 buggy_app 的生产者和消费者始终是两个线程，状态都正常。一个卡在死循环里空转的程序，在进程列表里和正常工作的程序长得一模一样——它不是在“等待”，而是在“忙等”，状态显示为 Ready，CPU 也在跑。

这一点我们是踩过坑才明白的。第一版 Skill 让 Agent 看进程列表判断系统状态，它把 buggy_app 的两个正常线程报成了“重复启动 bug”。

心跳增量是唯一无歧义的指标：数字涨了就是在干活，不涨就是卡住了，没有第三种解释。

报告

全程无人干预。Agent 自己读文件、执行命令、调用 LLM 分析、写报告。报告落盘到 `/data/ai_agent/evidence/report.md`，四段式：观察到的证据、假设、置信度、建议的下一步。

四段式是刻意的：把“看到了什么”和“我认为是什么”分开。LLM 容易把推断写成事实，强制它分栏陈述，人在复核时能一眼看出哪些是实测、哪些是猜测。

主线二：为 ESP32-P4 补上 ELF 动态加载

ESP32-P4 在 openvela 上此前无法执行编译期之外的程序。NuttX 自带的 `elf` 示例加载模块后，第一条指令就触发异常。

排查花了五个多小时，穿过两层根因：

第一层是内存保护。 PMP 把堆区标记为可读写、不可执行，而 ELF 模块的代码正是加载到堆上的——同一块内存，写得进去，取不出指令。定位手段是在板上把 16 条 PMP 配置寄存器读出来，确认那条规则的执行位是 0。

第二层是缓存一致性。 模块代码是经数据通路写入的（每次重定位都是一条 store），取指走另一条通路，而这颗芯片的 L1 缓存是写回模式——代码还留在数据缓存里，取指侧读到的是旧字节。定位手段是把加载后的内存 dump 出来，和主机上 objdump 的反汇编逐字节比对，证明内存内容正确、问题只能在取指路径。

沿途排除了五个假设，每一条都有实测依据而非推测（详见第四节）。

两处修复加起来不到二十行。修复后 elf 示例的七个测试全部通过：errno、hello、signal、struct、mutex、pthread、task，最后一个还在模块内部又 exec 了另一个模块。每个测试结束后内存收支归零，说明加载与卸载都是干净的，可以反复执行。

这是平台级的基础能力，不局限于本项目——任何需要在运行时加载插件、模块、用户程序的应用都用得上。

三、实测数据

连续运行 5.5 小时、162 轮无人干预，91% 完整执行七步，零缓存空转。

8 次失败原因与位置完全一致，是 404 KB 堆的已知限制，不是随机故障。

162 轮无人干预回归测试

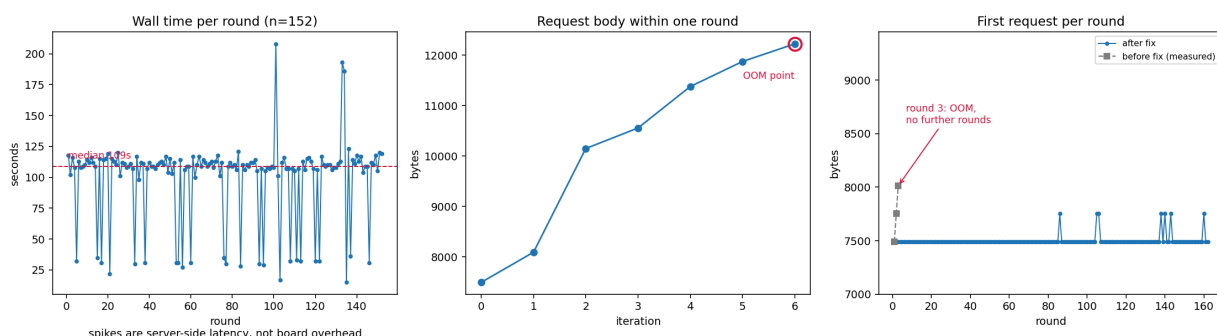
指标	数值
总轮数	162
成功	153 (94.4%)
其中完整执行七步	148 (91.4%)
失败	8 (4.9%)

指标	数值
缓存命中（空转）	0
单轮耗时	15s ~ 208s，中位 109s

口径说明：三个数字的统计口径不同，不能直接相加。

- 162 是心跳触发总数（日志中的 BEGIN）
- 153 是成功结束数（END status=ok）
- 8 是失败数（END status=fail）
- 差的 1 轮是拷贝日志时仍在进行中的那一轮

148 与 153 的区别：153 是成功结束的轮次，148 是其中 `iters=7 tools=7`、完整走完七步的。差的 5 轮成功但步数不足——模型少调了一个工具就给出结论，属正常波动。



图一 162 轮回归数据（左图 n=152 为有效耗时样本数）

左图是每轮的耗时。红色虚线标出中位数 109 秒，多数轮次贴着 100 到 120 秒。掉到 20 至 30 秒的那些是“成功但步数不足”的轮次——模型少调了一个工具就提前给出结论。两处冲到 200 秒的尖峰来自服务器响应延迟：我们做过同一时刻的对照，PC 与板子都是 8.2 秒，板端网络栈没有额外开销。

中图是一轮之内请求体的增长。横轴是七步中的第几步，纵轴是发给 LLM 的请求大小。每调用一个工具，结果都要追加进对话历史，所以从第 0 步的 7,492 字节涨到第 6 步的 12,502 字节。红圈标出的就是 OOM 发生的位置——那一步需要约 12.5 KB 的连续内存来序列化 JSON，404 KB 的堆已经凑不出来。8 次失败全部发生在这里。

右图是每一轮“第一次请求”的大小，最能说明缓存缺陷的影响。蓝色是修复后：162 轮的起点都是 7,492 字节，一条水平线。灰色方块是修复前的实测值，只有三个点——7,492、7,752、8,012——第三轮就因为会话历史累积而 OOM，没有第四轮。

关于那 8 次失败： `latency=0ms` 表明请求根本没发出去，是构造阶段失败而非网络问题。此时空闲总量仍有数十 KB，但已无足够大的连续块——是碎片而非耗尽。这是固有限制，第六节会讲为什么不打算绕开它。

一个日志看不出来的缺陷

这一节值得单独说，因为它决定了上面那组数据的可信度。

09-18 我们也跑过一批回归，114 轮，日志里每一行都是 `status=ok`。按那个数字，成功率接近 100%。

但统计工具调用次数时发现：**114 轮里只有 53 轮真正执行了诊断**，其余 57 轮的记录长这样：

```
[trace] END status=ok iters=1 tools=0 llm_ms=0 elapsed=0s
```

`llm_ms=0`、`tools=0` ——没有调用 LLM，没有执行任何工具，任务从未运行，而状态是成功。

根因是 LLM 响应缓存的键只有提示词的哈希。定时任务每次发送的提示词完全相同，一旦某一轮的回复进了缓存，之后每次触发都直接返回它。**日志显示一切正常，实际上系统已经空转了两个多小时。**

修复方式是让 `system` 通道不再查询缓存——周期检查的意义就在于观察系统此刻的状态，而“此刻的状态”本身就是不可缓存的。

	修复前	修复后
总轮数	114	162
真正执行诊断	53（46%）	148（91%）
缓存空转	57	0
按真实执行数计的失败率	7.5%	5.1%

失败的绝对数从 4 升到 8，但分母从 53 变成 156——修复前近半数轮次根本没有执行，自然不会失败。

这类缺陷读日志发现不了。每一行都显示正常，只有去核对“它到底调了几次工具”才会暴露。我们在写这份文档、核对数据时才发现它，然后重新采集了一批干净数据——就是上面那 162 轮。

正常与故障对照

状态	第一次心跳读数	第二次心跳读数	Agent 判定
正常	800	1000 (+200)	任务存活，报告 healthy
故障注入	50	50 (无变化)	任务停止前进

同一个 Skill、同一套七步流程，什么都没改，靠真实数据区分两种状态。

两份报告里的 ps 输出几乎相同——都是两个 buggy_app、都没有异常状态。区别只在心跳那一行。

四、关键技术难点与排查过程

第二节给了结论，这一节讲过程——五个小时里怎么一步步逼近，以及中间错在哪。

难处不在于两层根因本身有多深奥，而在于**起点几乎没有信息**。错误只有一行 MCAUSE=1，没有任何线索指向内存保护；关键开关 CONFIG_ESP_SYSTEM_MEMPROT 硬编码在第三方 HAL 的头文件里，在 NuttX 的 .config 里 grep 不到，我们最初因此断定 PMP 根本没被配置；而修好第一层之后错误码变成 MCAUSE=2，等于回到起点重来。

第一层：MCAUSE=1，取指被拒

加载 ELF 模块后触发异常：

```
riscv_exception: EXCEPTION: Instruction access fault.  
MCAUSE: 00000001, EPC: 4ff72840, MTVAL: 4ff72840
```

MCAUSE=1 是 RISC-V 的「取指被拒绝」。在此之前每一步都成功了：romfs 挂载、ELF 解析、段加载、符号绑定、数十次重定位写入全部正常，任务也创建了。**同一块内存可写不可执行**——只有第一条指令取不出来。

沿途排除五个假设

下面五条是按实际发生的顺序列的。每一条当时都像是答案，验证之后才知道不是——排除一条才想得到下一条。共同点是每条都有实测依据，不是推测。

1. **需要数据/指令总线地址转换。** ESP32-C3 的移植里有 `esp32c3_textheap.c`，把堆地址加一个偏移换成指令总线地址。查 P4 的 `soc.h`：`SOC_IRAM_LOW` 与 `SOC_DRAM_LOW` 同为 `0x4ff00000`——统一地址空间，不需要转换。排除。
2. **MMU 映射缺少执行权限。** 查 `esp_start.c` 中的 `mmu_hal_map_region` 调用，目标是 `MMU_TARGET_FLASH0`——这个 MMU 只映射外部 Flash，不涉及内部 SRAM。排除。
3. **入口地址计算错误。** REL 类型 ELF 的 `e_entry` 为 0，怀疑加载器算错了跳转目标。开启 `CONFIG_DEBUG_BINFMT` 后日志显示 `entrypt` 与 `textalloc` 一致，而 `e_entry=0` 对可重定位目标文件属正常。排除。
4. **ISA 不匹配。** `readelf -A` 对比模块与内核：两者同为 `rv32i2p1_m2p0_a2p1_c2p0_zicsr2p0_zifencei2p0`。排除。
5. **链接器松弛重定位处理有误。** 日志里满屏的 `RELAX` 记录引起怀疑。给测试模块加 `-mno-relax` 重新编译，`readelf -r` 确认 `RISCV_RELAX` 条目从 3 个降为 0——仍然崩溃。移除该参数后表现完全相同。排除。

转向硬件：板上读 PMP 寄存器

软件层面排除干净后转向硬件权限。在被测程序里加一个子命令，用内联汇编读取 PMP 配置寄存器并打印：

```
PMP05 cfg=8d R1 W0 X1 TOR L1 top=4ff49500
PMP06 cfg=8b R1 W1 X0 TOR L1 top=4ffc0000
```

PMP06 是 **RW 无 X**，失败的入口地址 `0x4ff72840` 正落在它的范围内。堆位于这个区间，模块代码加载到堆上，所以可写不可执行。

两条规则都带 L（lock）位——机器模式同样受限，且复位前无法修改。

根源是 `esp-hal` 的 `sdkconfig.h` 硬编码了 `CONFIG_ESP_SYSTEM_MEMPROT=1`。**该项不出现在 NuttX 的 `.config` 中，源码检索容易漏过**——我们最初 `grep` 不到它，因此误以为

PMP未被配置。它使 `cpu_region_protect.c` 以 `_iram_text_end` 为界把SRAM 切成 RX 与 RW 两段。

修复：将 PMP06 及其非缓存别名（entry 9）的权限从 RW 改为 RWX。

第二层：MCAUSE=2，指令无效

修复 PMP 后异常变为：

```
riscv_exception: EXCEPTION: Illegal instruction.  
MCAUSE: 00000002, EPC: 4ff72980, MTVAL: 00000000
```

错误码从 1 变成 2 本身就是进展：CPU 现在**被允许**取指了，只是读到的内容不是有效指令。

但这也意味着前面排除过的假设要重新审视——内容不对，可能是加载写错了，也可能是取指路径的问题。这次我们决定先看事实，不再猜。

在跳转前打印 `textalloc` 处的前 16 字节：

```
板上： 41 11 05 45 06 c6 26 c2 22 c4 97 00 0a f0 e7 80  
主机： 41 11 05 45 06 c6 26 c2 22 c4 97 00 00 00 e7 80
```

只有第 13、14 字节不同，而那正是 `auipc` 的重定位结果——**内存内容完全正确**。

问题只可能在取指路径。模块代码是经数据通路写入的（每次重定位都是一条 store），取指走另一条通路，而 P4 的内部内存经 L1 缓存访问且为写回模式

（`SOC_CACHE_INTERNAL_MEM_VIA_L1CACHE=1`），代码仍留在数据缓存中，取指侧读到旧字节。

实测 `fence.i` 单独不足。它排序的是访存与取指，不负责把写回缓存刷出——只加 `fence.i` 仍然复现。

最终修复：重定位完成后对代码段写回并失效。

```
cache_hal_writeback_addr(textalloc, textsize);  
cache_hal_invalidate_addr(textalloc, textsize);
```

小结

层	异常	定位手段	修复
一	MCAUSE=1 取指被拒	板上读 PMP 寄存器	堆区 RW → RWX
二	MCAUSE=2 指令无效	内存 dump 与 objdump 逐字节比对	写回 + 失效缓存

两处改动加起来不到二十行，但定位花了五个多小时。

五、向上游社区的贡献

开发过程中发现的缺陷，已有三项提交到 openvela 上游。

[nuttx#380](#)（对应补丁 0005）

ESP32-P4 上 `esp_hr_timer` 与 HAL 的 `esp_timer` 争用 SYSTIMER alarm 2 / TARGET2，导致以太网驱动无法初始化。我们在 issue 下补充了绕行方案：该定时器只用于周期轮询链路状态、不在数据通路上，改用 NuttX 的 `wdog` 实现即可避开争用。

[packages_ai_agent#37](#)（对应补丁 0009）

Skill 文件与文档硬编码 `/data/agent`，而 Kconfig 的默认值是 `/data/ai_agent`，读写不在同一位置。

[packages_ai_agent#39](#)（对应补丁 0009）

上一条的修复 PR，尚未合并。

尚未提交的部分

另有若干在真机上发现的缺陷，每一条的现象、根因、解法与验证记录都写在 `patches/README.md` 里（518 行）。其中几条不限于本项目——任何无 RTC、未开 RPMMSG、或在资源受限设备上部署 `ai_agent` 的项目都会遇到：

- **TLS 发送超时缺失**，连接池 stale 重连时永久阻塞，agent 主循环完全无响应

- 连接池复用已被服务器静默关闭的连接，请求在 TCP 重传中停滞约 100 秒，实测三次为 93.8 / 112.8 / 125 秒
- 定时任务被响应缓存永久短路，任务从不运行而日志显示 `status=ok`（第三节详述）
- HEARTBEAT.md 与 cron.json 创建路径与读取路径不一致，心跳与 cron 两个服务从未工作过
- 本地工具捷径截断多步推理，Skill 读入后流程在第一步就结束

这几条尚未整理成 issue，原因是比赛周期内优先保证了作品本身的完整性。补丁与验证记录都在仓库里，可以直接作为提交材料。

六、工程取舍与已知局限

这一节列出我们知道但没有解决的问题，以及为什么选择不解决。

Agent 只诊断不修复

Agent 的工具集里只有只读命令，没有杀进程、改配置、重启的能力。**这是刻意的限制。**

原因来自一次真实的误判：第一版 Skill 让 Agent 看进程列表判断系统状态，它把 buggy_app 的两个正常线程（生产者 + 消费者）报成了「重复启动 bug」，还建议「终止一个实例观察是否自动重启」。

如果它当时有修复权限，就会杀掉正常工作的程序。

当前策略是：Agent 负责采集、分析、写报告，由人决定下一步。

自动修复是可以考虑的方向，但前提是判据足够可靠——上面那次误判说明，仅凭进程列表做判断是不够的。心跳增量这个判据经过了 162 轮验证，在这个基础上再谈让 Agent 动手，才是安全的顺序。

PMP 权限的牺牲

将堆区从 RW 改为 RWX，牺牲了数据区不可执行的防护——堆上的数据理论上可以被当作代码执行。

正规方案是实现 `up_textheap_memalign()`，从内存中划出一块专用的可执行区域，只对那块设 RWX。该移植尚未提供这一支持，实现它需要改动链接脚本与内存布局。

我们也尝试过直接关闭 `CONFIG_ESP_SYSTEM_MEMPROT`，让整段 SRAM 变成 RWX，结果板子无法启动——该开关还牵连启动路径的其他分支。只改一条 PMP 规则是风险更小的选择。

堆碎片化

404 KB 堆在长时间运行后会碎片化，这是本项目最主要的限制，有两个可观察的后果。

其一是那 8 次失败。162 轮中失败 8 次，全部发生在第 7 步——写报告时的请求体是全流程最大的一次（约 12.5 KB），`cJSON` 序列化需要一块相应大小的连续内存。此时空闲总量仍有数十 KB，但已无足够大的连续块。

其二是更长时间后的 TLS 握手失败。实测连续跑一百多轮诊断后，空闲内存降到 34 KB，`ssl_setup` 返回 `0x7f00`，连 `net_test` 也一并失败。重启后恢复正常（空闲约 115 KB）。

当前的缓解方式是定期重启。减少报告内容或分段序列化也能降低单次分配的压力，但会增加实现复杂度。更根本的方案需要从内存分配策略或 Agent 的内存使用模式入手，超出了本次的范围。

七、仓库结构与复现方式

完整代码、补丁与全部原始数据都在参赛仓库 `contest2026_330_neiheyoudumuzhe`。

两档复现路径

快速验证只跑 ELF 动态加载，不需要 LLM 密钥和网络。按 README 的「运行方式」完成 repo sync、补丁应用、编译、烧录后，在板上执行 `elf`，预期七个测试依次通过，每个测试结束后打印 `Memory Usage ... Change: 0`。

完整演示需要配置 `packages/ai_agent/include/agent_secrets.h` 中的 LLM 地址与密钥（该文件不入库），重新编译烧录后：

```
nsh> buggy_app &          # 正常模式
nsh> ai_agent
    (等待启动完成, 出现 vela> 提示符)
vela> heartbeat_trigger # 手动触发一轮, 省去两分钟等待
```

注入故障则重启板子, 改用 `buggy_app stall &` 启动, 其余相同。

文档索引

文件	内容
README.md	目录结构、补丁清单、完整的运行步骤
patches/README.md	13 个补丁的现象、根因、解法、局限与验证记录
docs/technical-deep-dive.md	两个技术成果的完整排查过程
docs/evidence/	板上实测原始日志与统计脚本
docs/video-script.md	演示视频脚本与录制记录

文档中的每个数字都能在 `docs/evidence/` 里找到出处, 包括那些被排除的假设对应的日志。

关于 AI Coding 日志

本项目绝大部分开发使用网页版 Claude 完成。按大赛手册, 网页版不在官方支持的四种工具内, 无法自动采集对话日志, `logs/` 目录因此只覆盖收尾阶段。

不过排查过程本身留在了两处。 `patches/README.md` 逐个补丁记录了现象、根因、解法与局限; `docs/evidence/` 保留了每个结论对应的原始日志。ELF 加载那两层根因的定位过程——五个假设怎么一个个排除、PMP 寄存器读出来是什么、内存 dump 与主机 objdump 的逐字节比对——都能在这些材料里复核。